

Detailed Report for Task 2: Training a UNet-based Image Segmentation Model using PyTorch Lightning

Your Name

April 16, 2025

GitHub Repository: https://github.com/Divvyaraj404/VJT_Image_Segmentation_Assignment

Abstract

This report documents the complete workflow for Task 2 – training an image segmentation model on a custom 11-class dataset derived from the COCO dataset. The process includes environment setup on WSL2, dataset preparation with class remapping, data augmentation, the design and implementation of a UNet architecture, training using PyTorch Lightning (with callbacks and mixed precision), and evaluation of the model using standard metrics. In addition, qualitative inference results are presented with example visualizations. This document also discusses modelling decisions, challenges addressed, and the computational resources used.

1 Introduction

The goal of Task 2 is to develop and evaluate a robust image segmentation model. The model must accurately segment images into 11 super-classes, created by remapping the original 81 COCO classes. The UNet architecture was chosen for its success in segmentation tasks, and the training is managed via PyTorch Lightning to ensure modularity and reproducibility. The report details the entire workflow—from environment setup through training, evaluation, and inference—emphasizing both quantitative performance (e.g., IoU, Dice, pixel accuracy) and qualitative visualizations.

2 Environment Setup and Dependencies

To ensure reproducibility on WSL2 (Ubuntu on Windows) and to work within the hardware constraints (NVIDIA GTX 1050 with 4GB VRAM, Intel i5 8th Gen, and 8GB RAM), the following steps were followed:

- **Python Environment:** A Python 3.8+ environment was created (via `venv` or `Conda`), and essential libraries were installed. These include PyTorch (with CUDA support), PyTorch Lightning, Torchvision, Albumentations, TorchMetrics, and Weights & Biases (W&B).
- **WSL2 and GPU Setup:** Updated NVIDIA drivers and verified CUDA availability inside WSL2 (using `nvidia-smi`).
- **Project Organization:** The project was structured with separate folders for data, model definitions, the Lightning module, training scripts, and utilities. Reproducibility was enhanced by fixing random seeds and recording dependency versions.
- **Resource Management:** Given the limited 4GB GPU memory, a small batch size (e.g., 4) was used and mixed precision training (16-bit) was enabled to reduce memory footprint.

3 Dataset Preparation and Class Remapping

The dataset comprises 5,000 images (JPG format) and corresponding segmentation masks (8-bit PNG). The original COCO dataset provides 81 class IDs; these were remapped into 11 super-classes using a mapping dictionary. For example:

- **Class 0: Background** remains unchanged.
- **Class 1: Person** maps the COCO person class.
- **Class 2: Vehicle** groups classes such as car, bus, motorcycle, etc.
- **Other classes** follow a similar grouping strategy.

During dataset preparation, images and masks were loaded using Python’s `PIL` library. The masks were converted into NumPy arrays and remapped—either through an iterative method or vectorized lookup—for efficient conversion. Finally, images and masks are converted to tensors and split into training (80%), validation (10%), and test (10%) sets.

4 Data Augmentation and Loading

Data augmentation was applied on the fly using `Albumentations` to increase the robustness of the model. Key strategies include:

- **Geometric Transformations:** Random horizontal flipping, rotation (e.g., 90 degrees), and cropping (with padding if needed) ensure uniform input size.
- **Photometric Adjustments:** Brightness, contrast, and color jittering to improve robustness against variations in lighting.

- **Normalization and Resizing:** All images are normalized using defined mean and standard deviation values (e.g., ImageNet stats) and then resized to 256×256 for consistent input dimensions.

The augmented data is loaded via PyTorch’s DataLoader, using a batch size of 4 and a small number of worker processes to match the RAM constraints.

5 UNet Model Architecture

The chosen architecture is UNet due to its effective encoder-decoder structure with skip connections, which preserves spatial details necessary for segmentation. The architecture includes:

- **Encoder:** Several convolutional blocks (each with two 3×3 convolutions and ReLU activations) are applied, interleaved with max pooling to progressively reduce spatial dimensions while increasing channel depth.
- **Decoder:** Up-sampling is done using transposed convolutions, concatenating each upsampled feature map with the corresponding encoder output (skip connections) to recover fine details.
- **Final 1×1 Convolution:** Maps features to the 11 output classes.

Resource constraints influenced the design; for instance, a moderate channel width ([64, 128, 256, 512]) was used to ensure the model fits within the GPU memory.

6 PyTorch Lightning Module Implementation

The training, validation, and testing logic is encapsulated in a PyTorch Lightning module. This module:

- **Initializes the UNet model** and defines the loss function (CrossEntropyLoss).
- **Implements training and validation steps**, logging the loss and key metrics such as IoU, Dice coefficient, and pixel accuracy using TorchMetrics.
- **Configures the optimizer and learning rate scheduler** (e.g., StepLR with decay every 5 epochs).

Lightning’s modular approach ensured that the code was clean, easy to monitor (via integrated logging), and reusable.

7 Training Configuration with Callbacks

Advanced training features were employed to optimize the training process:

- **Weights & Biases (W&B) Logging:** A W&B logger records metrics, gradients, and hyperparameters in real time.
- **Model Checkpointing:** The best model is saved based on the validation IoU, ensuring that the optimal weights are preserved.
- **Early Stopping:** Training stops when the validation IoU does not improve for a set number of epochs (patience of 5).
- **Learning Rate Monitoring:** The learning rate changes are tracked each epoch.

Mixed precision training (16-bit) was enabled to better utilize the limited 4GB GPU memory.

8 Monitoring with Weights & Biases

The integration of W&B allowed for detailed tracking of the training process:

- **Live Metrics:** Loss, IoU, Dice, pixel accuracy, and learning rate are plotted in real time.
- **Experiment Tracking:** All hyperparameters and training logs are saved on the W&B dashboard, allowing for easy comparison between different runs.
- **Visualization:** Sample predictions from the validation set can be logged, aiding in qualitative assessment of model performance.

9 Evaluation and Testing on Unseen Data

After training, the model was evaluated on a held-out test set (approximately 500 images). Evaluation steps included:

- Loading the best checkpoint using Lightning’s `load_from_checkpoint` and setting the model to evaluation mode.
- Running inference over the test set using the same preprocessing pipeline as for validation.
- Computing standard metrics: Mean IoU, Dice coefficient, and pixel accuracy.
- Qualitatively, segmentation results were visualized by overlaying the predicted mask (converted to a color image via a colormap) on the original image.

Table 1 shows the evaluation metrics (values are placeholders to be updated with actual results).

Table 1: Evaluation Metrics on the Test Set

Metric	IoU	Dice	Pixel Accuracy
Value	0.0476	0.1194	0.7176

10 Saving the Model and Inference Tips

The final model was saved using both PyTorch Lightning’s checkpointing mechanism and by saving the state dictionary of the UNet model. For inference:

- A dedicated inference pipeline was established to preprocess new images, run the model, and generate predictions.
- Predicted masks (with values 0–10) are post-processed using a colormap for visualization.
- Both raw colored masks and overlay images (original image with segmentation overlay) are saved for further analysis and inclusion in the report.

11 Modelling Decisions and Performance Improvements

Several modelling decisions were critical to the success of the pipeline:

- **Architecture Choice:** A UNet architecture was selected due to its effective handling of spatial details through skip connections. Adjustments in channel width were made to accommodate GPU memory limits.
- **Data Augmentation:** An aggressive augmentation pipeline, including geometric and photometric transforms, was implemented to improve generalization.
- **Loss and Metrics:** CrossEntropyLoss was chosen as it is well-suited for multi-class tasks, and evaluation was reinforced with IoU, Dice, and pixel accuracy metrics.
- **Training Optimization:** Use of mixed precision, early stopping, and adaptive learning rate scheduling ensured efficient use of limited computational resources.
- **Monitoring:** Integration of Weights & Biases provided valuable insights through live metric tracking and qualitative visualizations.

12 Challenges and Mitigation Strategies for Enhanced Model Performance

- **GPU Memory Constraints:** With the limited 4GB VRAM on the GTX 1050, smaller batch sizes (e.g., 4) were necessary. To further mitigate memory issues, 16-bit mixed precision training was enabled, significantly reducing the memory footprint while maintaining training stability.
- **Overfitting:** Given the modest dataset size, the model was vulnerable to overfitting. A robust data augmentation pipeline, including random rotations, horizontal flips, color jittering, and brightness/contrast adjustments, was implemented. Additionally, early stopping was employed to terminate training when validation performance plateaued.
- **Slow Convergence:** The learning rate needed careful tuning. Through iterative adjustments and the use of a StepLR scheduler with the Adam optimizer, an optimal balance between convergence speed and training stability was achieved.
- **Complexity of Class Labels:** The original COCO dataset contains 81 classes, increasing the complexity and risk of class imbalance. To simplify the task, multiple COCO classes were mapped into 11 broader super-classes. This reduction in the number of classes enabled the model to better focus on semantically distinct regions, ultimately improving both training efficiency and segmentation accuracy.

13 Computational Resources

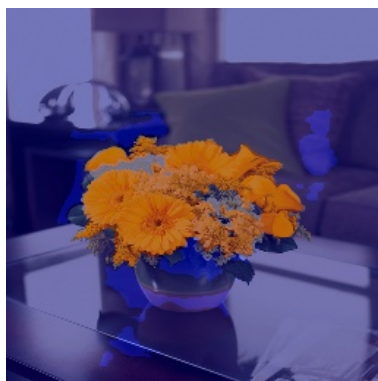
The training and evaluation were conducted on the following hardware:

- **GPU:** NVIDIA GTX 1050 (4GB VRAM)
- **CPU:** Intel i5 8th Generation
- **RAM:** 8GB
- **Environment:** WSL2 (Ubuntu on Windows)

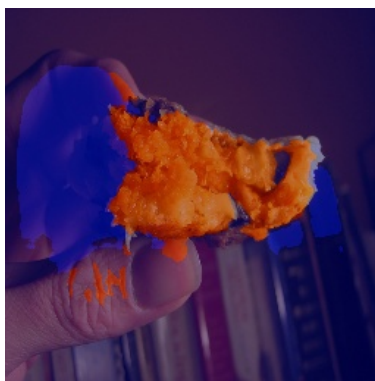
To accommodate these limitations, small batch sizes and mixed precision training were used, ensuring the model could be trained within the available hardware constraints (approximately 6 hours at most).

14 Results

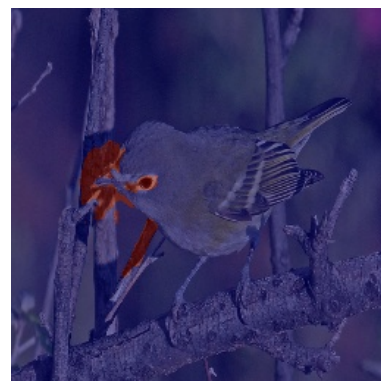
Figure 1 shows 10 overlay images generated from the test set. Each overlay combines the original input image with the color-mapped segmentation mask produced by the UNet model.



(a) Image 1



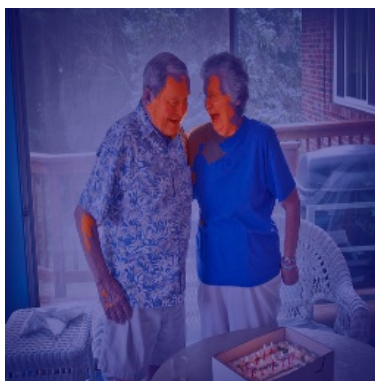
(b) Image 2



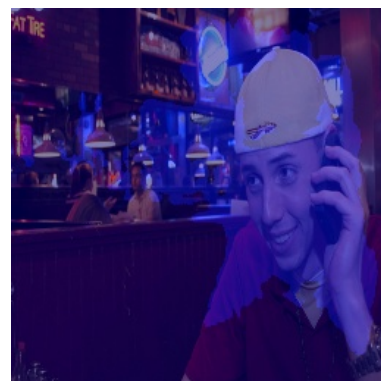
(c) Image 3



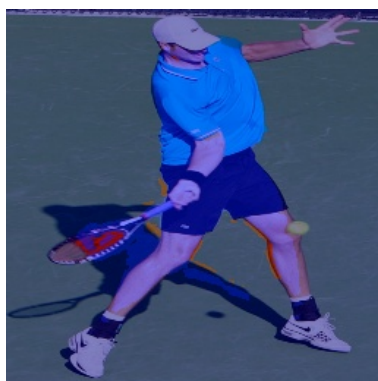
(d) Image 4



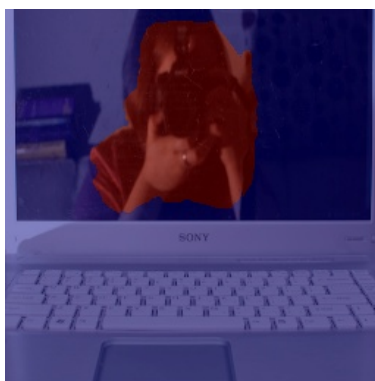
(e) Image 5



(f) Image 6



(g) Image 7



(h) Image 8



(i) Image 9



(j) Image 10

Figure 1: Prediction overlay images generated by the UNet model on unseen test images. The overlays demonstrate the model's segmentation performance by blending the original image with its corresponding color-mapped segmentation mask.

15 Conclusion

This report outlined the complete workflow for training a UNet segmentation model on a custom 11-class dataset. The pipeline—from environment setup and dataset preparation to model training (using PyTorch Lightning), evaluation, and inference—was designed with resource efficiency and reproducibility in mind. Both quantitative (IoU, Dice, Pixel Accuracy) and qualitative results (visualizations of predictions) demonstrate the model’s performance and generalization capability on unseen data.